CIIRS

Smart CCTV Incident Intelligence & Service Management System

A Project Report

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

Computer Science and Engineering

Submitted by:

Agam Thakur (UID: 25BCS11039)

Under the Guidance of:

Monika Bishnoi

Institute of Engineering (UIE)

Department of Computer Science & Engineering



Chandigarh University

November 2026

Table of Contents

S. No.	Content	Page No.
1	Introduction and Literature Review	3
2	Problem Definition and Objectives	4-5
3	Hardware and Software Used	6-7
4	Implementation	8-21
5	Results and Discussion	22-23
6	Conclusion and Future Scope	24
7	References	25

1. Introduction and Literature Review

Educational institutions and municipal campuses manage several daily activities, including student complaints, safety incidents, maintenance requests, facility reservations, and administrative communication. When these activities are handled manually or through separate communication channels, students may experience delays in receiving updates, while administrators may find it difficult to maintain records and coordinate requests.

CampusConnect / CIIRS is a Smart Campus Service Management and Incident System developed using Java. It provides a centralized platform through which students and citizens can submit complaints, book campus resources, track request statuses, and receive notifications. Administrators and operators can manage complaints, monitor reservations, update resource information, and oversee campus activities.

The project follows an object-oriented approach and uses Java programming concepts such as classes, objects, inheritance, abstraction, polymorphism, exception handling, file handling, serialization, multithreading, synchronization, and JDBC connectivity.

The application follows a layered architecture using Spring Boot and Spring MVC. The presentation layer provides the user interface, the controller layer handles requests, the service layer implements business logic, and the repository layer communicates with the database.

1.1 Literature Review

Java is widely used for developing secure, scalable, and platform-independent applications. Object-oriented programming allows software to be divided into reusable classes and objects. Java exception handling improves application reliability, while its I/O libraries support file processing and data storage.

Multithreading allows multiple tasks to execute concurrently, improving responsiveness. Synchronization helps prevent race conditions when multiple users access shared resources. JDBC provides connectivity between Java applications and relational databases.

Spring Boot simplifies enterprise application development by providing auto-configuration, dependency management, and integration with Spring MVC.

CampusConnect combines these concepts to create a centralized campus service platform.

2. Problem Definition and Objectives

2.1 Problem Definition

Traditional campus service management often depends on manual registers, emails, and separate communication channels. This creates difficulties in tracking complaints, managing reservations, maintaining records, and communicating updates.

Students may not know the current status of their complaints, while administrators may find it difficult to handle multiple requests efficiently. Resource booking can also create conflicts when two or more students attempt to reserve the same facility for overlapping time slots.

Therefore, a centralized system is required to manage campus services efficiently, maintain organized records, and reduce manual work.

2.2 Objectives

The main objectives of CampusConnect are:

1. To develop a centralized campus service management platform using Java and Spring Boot.
2. To implement object-oriented programming principles such as encapsulation, inheritance, abstraction, and polymorphism.
3. To provide role-based access for students and administrators.
4. To allow students to submit complaints and track their status.
5. To provide a resource booking facility with conflict checking.
6. To implement exception handling for errors and invalid operations.
7. To demonstrate file handling and object serialization.
8. To use multithreading for background notification processing.
9. To implement synchronization to manage concurrent booking requests.
10. To establish database connectivity using JDBC and Spring JdbcTemplate.

2.3 Scope of the Project

Student Operations

- Student login and dashboard.
- Complaint submission and tracking.
- Campus resource booking.
- Booking history.
- Notification viewing.

Administrator Operations

- Administrative dashboard.
- Complaint management.
- Complaint status updates.
- Resource management.

- Booking monitoring.
- Backup and restore operations.

Technical Scope

The project demonstrates Java OOP concepts, custom exceptions, file handling, serialization, multithreading, synchronization, JDBC, and Spring Boot MVC.

3. Hardware and Software Used

3.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent recommended
RAM	Minimum 4 GB; 8 GB recommended
Storage	Approximately 2 GB free space
System	Desktop or laptop computer

3.2 Software Requirements

Software	Purpose
Operating System	Windows 10/11, Linux or macOS
Java JDK 17	Application development
Spring Boot 3.2.5	Backend framework
Spring MVC	Request handling
MySQL 8.0 / H2	Database management
JDBC	Database connectivity
Spring JdbcTemplate	Database operations
Thymeleaf	Dynamic web pages
Bootstrap 5	Frontend styling
Apache Maven	Dependency and build management
VS Code / IntelliJ IDEA	Development environment

3.3 Technologies Used

Java 17: Used for object-oriented programming, exception handling, file handling, serialization, and multithreading.

Spring Boot: Used to develop and configure the backend application.

Spring MVC: Handles HTTP requests and connects the frontend with backend services.

JDBC: Establishes database connectivity and executes SQL queries.

MySQL: Stores student information, complaints, bookings, resources, and notifications.

Thymeleaf and Bootstrap: Used to create dynamic and responsive user interfaces.

4. Implementation

4.1 System Architecture

CampusConnect follows a layered architecture that separates the user interface, request handling, business logic, and database operations.

The architecture consists of the following layers:

1. Presentation Layer: Uses Thymeleaf templates and Bootstrap 5 to display the user interface.
2. Controller Layer: Handles HTTP requests through Spring MVC controllers.
3. Service Layer: Contains the main business logic for authentication, complaints, bookings, resources, and notifications.
4. Threading and I/O Layer: Handles background notification processing, file storage, serialization, and booking synchronization.
5. Repository Layer: Communicates with the database using JDBC and JdbcTemplate.

Architecture Flow:

User Browser → Spring MVC Controller → Service Layer → Java I/O and Threading Utilities → JDBC Repository → Database.

4.2 Main Modules

1. Authentication Module

This module validates login credentials and provides access according to the user's role. Students and administrators are redirected to their respective dashboards.

2. Student Module

Students can submit complaints, attach files, book campus resources, check booking history, and view complaint status.

3. Administrator Module

Administrators can view complaints, update their status, manage resources, and monitor bookings.

4. Complaint Management Module

This module manages complaints through different stages, including PENDING, IN_PROGRESS, RESOLVED, and REJECTED.

5. Resource Booking Module

Students can request campus resources such as laboratories and seminar halls. The system checks for overlapping reservations before confirming a booking.

6. Notification Module

Notifications are processed through a background worker thread so that notification processing does not unnecessarily delay the main request.

7. File Management Module

This module handles complaint attachments and supports binary backup and restoration using Java serialization.

4.3 Implementation of Object-Oriented Programming Concepts

Object-oriented programming is one of the main foundations of CampusConnect. The project uses classes and objects to represent users and campus operations.

A. Class and Object

A class defines the properties and behaviour of an object. In CampusConnect, User represents a common user structure.

```Java

public class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public void displayDetails() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Email: " + email);
    }
}
```

Object Creation:

```Java

public class Main {
    public static void main(String[] args) {
```

```java
        User user = new User(
            101,
            "Agam",
            "student@campusconnect.com"
        );

        user.displayDetails();
    }
}
```

B. Encapsulation

Encapsulation is the process of wrapping data and methods into a single unit. In Java, it is achieved by declaring variables private and providing public getter and setter methods.

```java
public class Complaint {
    private int complaintId;
    private String description;
    private String status;

    public void setStatus(String status) {
        this.status = status;
    }

    public String getStatus() {
        return status;
    }
}
```

In CampusConnect, encapsulation protects important information such as complaint status and user details from direct modification.

C. Inheritance

Inheritance allows one class to acquire the properties and methods of another class.

CampusConnect uses a common User class, which can be extended by Student and Admin classes.

```java
public class User {
```

```java
    protected String name;
    protected String email;

    public void login() {
        System.out.println("User logged in");
    }
}

class Student extends User {
    public void submitComplaint() {
        System.out.println("Complaint submitted");
    }
}

class Admin extends User {
    public void manageComplaints() {
        System.out.println("Managing complaints");
    }
}
```

The Student and Admin classes reuse the common properties and methods of User.

D. Method Overriding and Polymorphism

Method overriding occurs when a child class provides its own implementation of a method defined in its parent class.

```java
class User {
    public String getDashboardRoute() {
        return "/dashboard";
    }
}

class Student extends User {
    @Override
    public String getDashboardRoute() {
        return "/student/dashboard";
    }
}
```

```java
class Admin extends User {
    @Override
    public String getDashboardRoute() {
        return "/admin/dashboard";
    }
}
```

Runtime Polymorphism:

```java
public class Main {
    public static void main(String[] args) {
        User user1 = new Student();
        User user2 = new Admin();

        System.out.println(user1.getDashboardRoute());
        System.out.println(user2.getDashboardRoute());
    }
}
```

The method executed depends on the actual object created at runtime.

E. Method Overloading

Method overloading allows multiple methods to have the same name but different parameter lists.

```java
class ComplaintService {
    public void submitComplaint(String description) {
        System.out.println("Complaint: " + description);
    }

    public void submitComplaint(
        String description,
        String priority
    ) {
        System.out.println(
```

```
Java

            "Complaint: " + description +
            ", Priority: " + priority
        );
    }
}
```

This allows complaints to be submitted with or without priority information.

F. Interface

An interface defines a contract that implementing classes must follow.

```
Java

interface NotificationService {
    void sendNotification(String message);
}

class NotificationServiceImpl
    implements NotificationService {

    @Override
    public void sendNotification(String message) {
        System.out.println("Notification: " + message);
    }
}
```

This interface separates notification functionality from its implementation.

4.4 Exception Handling

Exception handling is used to manage runtime errors and prevent unexpected application termination. CampusConnect uses custom exceptions for booking conflicts and other application-specific errors.

Custom Exception:

```
Java

class CampusConnectException
    extends RuntimeException {

    public CampusConnectException(String message) {
```

```java
        super(message);
    }
}

class BookingConflictException
    extends CampusConnectException {

    public BookingConflictException(String message) {
        super(message);
    }
}
```

Using the Custom Exception:

```java
public class BookingService {
    public void checkBooking(boolean available) {
        if (!available) {
            throw new BookingConflictException(
                "Resource is already booked!"
            );
        }
        System.out.println("Booking successful");
    }
}
```

This prevents users from booking an already reserved resource when the availability check detects a conflict.

4.5 File Handling and Java I/O

Java provides input and output streams for reading and writing files. CampusConnect uses file handling for complaint attachments and backup operations.

Example: Writing Data to a File

```java
import java.io.*;
```

```java
public class FileHandlingDemo {
    public static void main(String[] args) {
        try (
            FileOutputStream file =
                new FileOutputStream("complaint.txt");
            BufferedOutputStream buffer =
                new BufferedOutputStream(file)
        ) {
            String data =
                "Campus maintenance complaint";
            buffer.write(data.getBytes());
            System.out.println("File created successfully");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Buffered streams improve file operations by reducing the number of direct interactions with the underlying file.

4.6 Object Serialization

Serialization converts an object into a byte stream so that it can be stored in a file or transferred. CampusConnect uses serialization for binary backup and restoration of complaint data.

Serializable Complaint Class:

```java
import java.io.Serializable;

public class Complaint implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String description;
```

```java
    private String status;

    public Complaint(
        int id,
        String description,
        String status
    ) {
        this.id = id;
        this.description = description;
        this.status = status;
    }
}
```

Serialization Example:

```java
import java.io.*;

public class BackupManager {
    public static void main(String[] args) {
        Complaint complaint =
            new Complaint(
                101,
                "Projector not working",
                "PENDING"
            );

        try (
            ObjectOutputStream out =
                new ObjectOutputStream(
                    new FileOutputStream("backup.ser")
                )
        ) {
            out.writeObject(complaint);
            System.out.println("Backup created successfully");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

The ObjectOutputStream writes the object into a binary file. ObjectInputStream can later be used to restore the serialized object.

4.7 Multithreading

Multithreading allows multiple tasks to execute concurrently. CampusConnect uses background processing for notifications.

The notification worker processes queued tasks separately from the main request-handling flow.

Example:

```java
class NotificationTask implements Runnable {
    private String message;

    public NotificationTask(String message) {
        this.message = message;
    }

    @Override
    public void run() {
        System.out.println(
            "Processing notification: " + message
        );
    }
}

public class Main {
    public static void main(String[] args) {
        Thread worker = new Thread(
            new NotificationTask(
                "Your complaint has been updated"
            )
        );
        worker.start();
    }
}
```

The start() method creates a separate execution thread, which runs the code inside the run() method.

4.8 Thread Synchronization

When multiple students attempt to book the same resource at the same time, a race condition may occur.

Synchronization ensures that a critical section is accessed safely.

Example:

```java
class BookingManager {
    private boolean booked = false;

    public synchronized void bookResource() {
        if (!booked) {
            booked = true;
            System.out.println(
                "Resource booked successfully"
            );
        } else {
            System.out.println(
                "Resource already booked"
            );
        }
    }
}
```

The synchronized keyword ensures that only one thread at a time can execute this method on the same BookingManager instance.

The project report also describes the use of ReentrantLock for coordinating booking operations.

4.9 JDBC Database Connectivity

JDBC allows Java applications to connect to relational databases and execute SQL queries.

CampusConnect uses JDBC and Spring JdbcTemplate for database operations.

Example: JDBC Connection

```Java

import java.sql.*;

public class DatabaseConnection {
    public static void main(String[] args) {
        String url =
            "jdbc:mysql://localhost:3306/campusconnect";
        String username = "root";
        String password = "your_password";

        try {
            Connection con =
                DriverManager.getConnection(
                    url,
                    username,
                    password
                );
            System.out.println(
                "Database connected successfully"
            );
            con.close();
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Example: PreparedStatement

```Java

String sql =
    "SELECT * FROM resources WHERE available = ?";

try (
    Connection con =
        DriverManager.getConnection(url, username, password);
    PreparedStatement ps =
```

```java
        con.prepareStatement(sql)
) {
    ps.setBoolean(1, true);
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        System.out.println(
            rs.getString("name")
        );
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

PreparedStatement uses parameterized queries and helps protect against SQL injection.

4.10 Spring Boot Integration

Spring Boot is used to simplify backend development. It supports dependency injection, auto-configuration, and Spring MVC request handling.

Example: Spring Boot Controller

```java
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class StudentController {

    @GetMapping("/student/dashboard")
    public String dashboard() {
        return "student-dashboard";
    }
}
```

When a student accesses the dashboard URL, Spring MVC maps the request to the controller method and returns the corresponding Thymeleaf page.

4.11 System Workflow

The main execution workflow of CampusConnect is:

1. The user opens the CampusConnect application.
2. The user enters login credentials.
3. The authentication module validates the credentials.
4. The user is redirected to the appropriate dashboard.
5. Students submit complaints or request campus resource bookings.
6. The booking module checks resource availability.
7. The system stores valid requests in the database.
8. Notifications are added to the background processing queue.
9. Administrators review complaints and update their status.
10. The system maintains records and supports backup operations.

5. Results and Discussion

5.1 Results

CampusConnect is designed to provide a centralized platform for managing student complaints, campus resources, and administrative activities.

The following test cases are included in the original project report.

Test ID	Test Case	Expected Result	Status
TC-01	Valid student login	Student dashboard opens	**Verified**
TC-02	Invalid login credentials	Authentication rejected	**Verified**
TC-03	Complaint submission with attachment	File stored and complaint saved	**Verified**
TC-04	Admin complaint status update	Status updated and audit entry logged	**Verified**
TC-05	Available resource booking	Booking confirmed	**Verified**
TC-06	Concurrent overlapping booking	Booking conflict detected	**Verified**
TC-07	Student accesses admin page	Unauthorized access denied	**Verified**
TC-08	Object serialization & deserialization	Backup file created and restored	**Verified**
TC-09	Asynchronous notification processing	Notification processed by worker thread	**Verified**
TC-10	JDBC transaction error and rollback	Database consistency maintained	**Verified**

Note: These test results reflect the validation suite performed as part of the system verification phase.

5.2 Discussion

The project demonstrates the practical application of Java programming concepts in a campus management environment.

Object-oriented programming provides a structured way to manage different user roles. Exception handling improves error management, while Java I/O and serialization support file operations and backup functionality.

Multithreading allows notifications to be processed in the background. Synchronization helps manage simultaneous booking requests and reduces the possibility of conflicting reservations within the application instance.

JDBC provides communication between Java and the relational database, while Spring Boot simplifies the development of the web application.

5.3 Advantages

1. Centralized campus service management.
2. Better complaint tracking and administration.
3. Organized resource reservation.
4. Reduced manual work.
5. Role-based access for students and administrators.
6. Background notification processing.
7. Structured database management.
8. Demonstration of important Java programming concepts.

5.4 Limitations

1. Booking synchronization within a single application instance does not automatically coordinate multiple deployed instances.
2. External SMS and email notification services are not included in the described implementation.
3. The application requires a working database connection.
4. Further security and performance testing is required before large-scale deployment.

6. Conclusion and Future Scope

6.1 Conclusion

CampusConnect: Smart Campus Service Management System demonstrates the practical use of Java and enterprise Java technologies to solve campus management problems.

The project combines object-oriented programming, exception handling, file handling, object serialization, multithreading, synchronization, JDBC connectivity, and Spring Boot MVC into a centralized application.

The system provides a structured approach to managing student complaints, campus resource reservations, notifications, and administrative activities.

Through this project, important programming concepts are applied to a practical scenario. It also demonstrates how Java can be used to develop modular and maintainable applications.

6.2 Future Scope

The project can be improved in several ways:

1. Email and SMS Integration: External notification services can be integrated to send updates directly to students.
2. University Single Sign-On: OAuth2 or LDAP authentication can be implemented for university account integration.
3. QR Code Booking Verification: QR codes can be introduced for checking in to laboratories and seminar halls.
4. Cloud Deployment: The application can be deployed on cloud infrastructure to improve availability and scalability.
5. Distributed Synchronization: Redis-based distributed locking can be implemented to coordinate bookings across multiple application instances.
6. Mobile Application: A mobile application can be developed to provide easier access to campus services.

7. References

[1] Oracle, “Java Platform, Standard Edition 17 Documentation.” [Online]. Available: https://docs.oracle.com/en/java/javase/17/

[2] Spring, “Spring Boot Reference Documentation.” [Online]. Available: https://docs.spring.io/spring-boot/

[3] Spring, “Spring Framework Reference Documentation.” [Online]. Available: https://docs.spring.io/spring-framework/reference/

[4] Oracle, “JDBC Basics Tutorial.” [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/

[5] MySQL, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/

[6] Apache Maven, “Maven Documentation and Guides.” [Online]. Available: https://maven.apache.org/guides/

[7] J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.